Member 2 — Hardware, Sensors and Measurement Reliability

Member placeholder: [MEMBER 2 NAME]

Proposed viva responsibility: installed hardware, wiring/bus topology, TMP117 protocol and raw diagnostic, MAX30102 optical-quality guards, MPU scaling/calibration limits, and the boundary between actual sensor temperature and clinical wrist temperature.

Authorship disclaimer: this is a proposed responsibility allocation for the HOD discussion. The repository proves that the hardware paths exist; it does not prove that [MEMBER 2 NAME] personally wrote them. The team must confirm authorship. Use **"the team implemented"** for source-backed shared work.

A. Responsibility scope

A.1 One-sentence scope

For this discussion, [MEMBER 2 NAME] is responsible for explaining what is physically installed, how it is wired, what the raw sensor values mean, how invalid readings are rejected, and exactly which hardware claims remain unverified.

A.2 Included in this responsibility

- The installed component list and explicit non-installed modules.
- Power, ground, address strapping and bus topology for Wire and Wire1.
- TMP117 register map, signed conversion, configuration, identity check and raw diagnostic.
- TMP117 stuck-bus and abrupt-change protection.
- Actual TMP117 sensor temperature versus clinical wrist temperature at the hardware boundary.
- MAX30102 initialization, sample-quality guards, HR/SpO2 acceptance limits and limitations.
- MPU-6050/MPU-6500 identity, ranges, raw conversion, startup calibration limits and fall candidate inputs.
- Interpretation of sensor ranges: accepted code range is not the same as measured accuracy.

- Physical verification status and what a reference measurement would need to show.

A.3 Outside primary scope — but required knowledge

- Firmware scheduling and JSON assembly: Member 1.
- Serial parser/API: Member 3.
- SQLite/dashboard storage and display: Member 4.
- Baseline statistics and deterministic alert policy: Member 5.

Member 2 must still know that the bridge and dashboard independently re-check the raw temperature evidence; hardware validity is not established by a single layer.

B. Achievements attributable to this area

Achievement	**What exists**	**Evidence**
Documented installed inventory	OLED, MAX30102, TMP117, MPU, SOS; SHT40/MLX explicitly absent	[SOURCE] README.md, wiring-guide.html, src/main.cpp
Separate IMU bus	Wire1 on GPIO 17/18 at 400 kHz; primary Wire on GPIO 8/9 at 50 kHz	[SOURCE] src/main.cpp
Checked TMP117 protocol	Identity 0x0117, config 0x01A0, writable-mask readback, offset readout	[SOURCE] src/tmp117_protocol.h, initTmp117()
TMP117 raw conversion tests	Compile-time vectors plus tests/test_tmp117_vectors.py	[TEST]
Stuck-bus rejection	0x0000 and 0xFFFF rejected; -0.01 °C all-ones anomaly explained as a hypothesis	[SOURCE] + [TEST]
Raw diagnostic	Separate PlatformIO project scans, configures and logs raw values/failures	[SOURCE] diagnostics/tmp117_raw/; [BUILD]
Optical quality guard	DC/AC/perfusion thresholds before HR/SpO2 are accepted	[SOURCE] sensor_math.h, finishOpticalWindow()
MPU scale/range implementation	±8 g, ±250 dps, 4096 LSB/g, 131 LSB/dps	[SOURCE] + compile-time assertions

Achievement	What exists	Evidence
Honest calibration boundary	Startup bias is single-orientation and not six-face calibration	[SOURCE] collectMpuCalibrationWindow()
Conservative clinical temperature	Wrist value/raw/timestamp null while contact is unverified; no fixed offset	[SOURCE] + [TEST]

B.1 What is not an achievement yet

- Confirming the actual TMP117 address/identity/config on the assembled board.
- Capturing the -0.01 °C event with raw bytes.
- Reference-thermometer accuracy or uncertainty for TMP117.
- MAX30102 HR/SpO2 accuracy under real placement, motion or poor perfusion.
- MPU axis/sign/range validation, six-face calibration or fall-detector sensitivity.
- OLED, SOS or fall tests on physical hardware.
- Any SHT40 or MLX90614 test — neither device is installed.

C. Detailed code walkthrough

C.1 Installed hardware and wiring

Device	Interface	Address	Wiring / notes
ESP32-S3 DevKitC-1 N16R8	—	—	16 MB QIO flash, 8 MB OPI PSRAM; custom board manifest
SSD1306 128×64	Wire	0x3C	3.3 V, GND, SDA GPIO 8, SCL GPIO 9
MAX30102	Wire	0x57	3.3 V, GND, SDA GPIO 8, SCL GPIO 9
7SEMI TMP117 ES-12132	Wire	0x48–0x4B	VIN 3.3 V only; GND; SDA 8; SCL 9; ADD selects address; ALT unused

Device	Interface	Address	Wiring / notes
MPU-6050 / MPU-6500	Wire1	0x68 or 0x69	3.3 V, GND, SDA GPIO 17, SCL GPIO 18; AD0 selects address
SOS button	GPIO	GPIO 5	To GND, internal pull-up, active LOW
SHT40	—	—	**Not installed**, compiled out
MLX90614	—	—	**Not installed**, compiled out

The primary bus is initialized at 50 kHz (I2C_PRIMARY_HZ) and the IMU bus at 400 kHz (I2C_IMU_HZ). Both buses set a 25 ms Wire timeout.

Naming boundary: the dashboard must keep the actual sensor separate as **TMP117 Sensor Temperature (°C)** and the clinical metric as **Wrist Skin Temperature (°C)**. A hardware reading must never be copied from the actual field into the wrist field. Core remains unavailable, fever remains not_available, and the exact message is **Core temperature unavailable — confirmation required**.

Electrical cautions from the wiring documentation:

- TMP117 supply goes to VIN at 3.3 V; do not use 5 V and do not jumper the separate 3.3 V pin to VIN.
- Common ground is mandatory.
- TMP117 ADD selects one of 0x48–0x4B; the firmware verifies identity rather than assuming an address.
- TMP117 ALT is not implemented.
- MPU AD0 must be tied to GND (address 0x68) or deliberately set for 0x69; leaving it floating can cause address instability. The firmware checks WHO_AM_I 0x68 or 0x70.
- If a different fixed-address device such as a MAX30205 were added at 0x48, the TMP117 strapping would need to avoid that conflict. No MAX30205 is installed in this build.

C.2 TMP117 protocol — src/tmp117_protocol.h

The header cites the TI TMP117 datasheet for the temperature result and identity registers.

Constant	Value	Meaning
REG_TEMPERATURE	0x00	Temperature result
REG_CONFIGURATION	0x01	Configuration
REG_T_HIGH_LIMIT	0x02	High limit (not used as an alert here)
REG_T_LOW_LIMIT	0x03	Low limit (not used as an alert here)
REG_TEMPERATURE_OFFSET	0x07	Offset register
REG_DEVICE_ID	0x0F	WHO_AM_I
DEVICE_ID_TMP117	0x0117	Expected identity
LSB_C	0.0078125	°C per signed LSB
CONFIG_CONTINUOUS_8AVG_500MS	0x01A0	Continuous, 8-sample average, 500 ms cycle
CONFIG_WRITABLE_MASK	0x0FE0	Writable configuration bits
CONFIG_DATA_READY_MASK	0x2000	Data-ready status bit
CONFIG_EEPROM_BUSY_MASK	0x1000	EEPROM-busy status bit
MIN_SENSOR_C / MAX_SENSOR_C	-55 / 150	Accepted actual-sensor range in code

Conversion:

```
constexpr float celsiusFromRawCode(uint16_t raw) {
  return static_cast<float>(static_cast<int16_t>(raw)) * LSB_C;
}
```

The int16_t cast is essential. If the value were treated as unsigned, 0xFFFF would become a large positive value instead of -0.0078125 °C. Compile-time assertions cover:

- big-endian byte order 0x0E,0xD3 → 0x0ED3;
- 0xFFFF → -0.0078125;
- 0x0ED3 → 29.6484375;
- 0x8000 → -256.0;
- stuck patterns 0x0000 and 0xFFFF;
- range boundaries -55 and 150, and rejection just outside.

C.3 TMP117 initialization — initTmp117()

initTmp117():

1. Sets the primary bus to 50 kHz.
2. Clears status/value/raw/offset state.
3. Scans {0x48, 0x49, 0x4A, 0x4B}.
4. Requires an ACK and reads two ID bytes big-endian.
5. Requires id == 0x0117; it does not accept an address merely because something responds.
6. Reads configuration before writing it.
7. Writes 0x01A0.
8. Delays 5 ms, reads back the configuration and compares the 0x0FE0 writable mask.
9. Reads offset register 0x07; reports raw/signed/°C and warns on nonzero without applying a software correction.
10. Sets tmp117Present, warming_up, clears error counters, waits 600 ms for the first 500 ms conversion, and calls readTmp117().

If no candidate has the correct identity/config, it sets not_available with not_found_or_identity_mismatch. A configured part with nonzero offset is not "corrected"; the offset is diagnostic information.

C.4 TMP117 read sequence — readTmp117()

The order is deliberate because reading the configuration register clears DATA_READY:

if not present -> not_available

read configuration (also clears DATA_READY)

if EEPROM busy -> warming_up / eeprom_busy

if DATA_READY == 0 -> warming_up / data_not_ready

read temperature register

if raw is 0x0000 or 0xFFFF -> sensor_error / stuck_bus_raw_pattern

convert signed code

if outside -55..150 °C -> sensor_error / outside_sensor_range

if jump > 5 °C from last accepted value:

    require a confirming sample within 0.5 °C; otherwise low_quality pending

accept: last accepted = new value; valid; timestamp = now

Important consequences:

- A stuck bus cannot be accepted as a near-zero temperature.
- A real reading outside the clinical wrist range but inside -55..150 °C can still be a valid actual sensor reading. It is not promoted to wrist.
- A large step is not immediately accepted; it needs one confirmation sample.
- On an address NACK, the firmware marks the device absent (not_available); other I2C errors produce sensor_error.
- updateTmp117() reads every 2 s, slower than the 1 s telemetry cadence. The same last accepted value may be republished with the last valid timestamp, provided freshness is within 6 s; tmp117ValueValid() also requires a valid status.

C.5 TMP117 raw diagnostic — diagnostics/tmp117_raw/

The diagnostic is a separate PlatformIO project. It deliberately avoids the Adafruit TMP117 library so an I2C failure cannot be mistaken for a temperature.

It:

- prints the primary bus, GPIO 8/9 and 50 kHz;
- scans the bus and identifies 0x3C (OLED) and 0x57 (MAX30102) as unrelated;
- searches 0x48–0x4B and prints WHO_AM_I, flagging 0x0117 as confirmed;
- reads configuration before the change;
- writes continuous/8-average/500 ms;
- prints readback and whether the writable-mask match succeeds;
- reads offset 0x07 raw/signed/°C;

- once per second prints address, config, drdy, raw MSB/LSB, signed code and temperature;
- flags ALL_ONES, ALL_ZEROS, outside-range and read errors;
- prints temp_read=ERR(...) <-- no conversion; value is NOT zero;
- prints a summary every 30 s with successful reads, failures, all-ones, all-zeros, data-not-ready and min/max temperature;
- retries the scan every 5 s if the sensor disappears.

The intended capture is at least five minutes, including the -0.01 °C event. **No such capture was made in the 2026-10-29 documentation pass.**

C.6 MAX30102 path

initMax30102():

- calls max30102.begin(Wire, I2C_SPEED_STANDARD);
- reads PART_ID and requires 0x15;
- calls max30102.setup(0x24, 4, 2, 100, 411, 4096);
- sets red/IR pulse amplitude 0x24;
- clears FIFO and starts in warming_up.

sampleOptical():

- if more than 150 ms pass between samples, resets optical state, clears FIFO and marks low quality;
- reads FIFO IR/red pairs;
- rejects IR or red < 50000 as no contact;
- rejects IR or red >= 262143 as saturation/low quality;
- accumulates 100 samples;
- calls finishOpticalWindow().

finishOpticalWindow() computes minimum, maximum, mean, AC and perfusion index, then:

bool qualityGood = sensor_math::maxSignalQuality(mean, ac, pi);

`bool hrAccepted = qualityGood && bpmValid && bpm >= 30 && bpm <= 240;`

`bool spo2Accepted = qualityGood && spo2Valid && spo2 > 0 && spo2 <= 100;`

Quality thresholds in `sensor_math.h` are engineering thresholds, not clinical thresholds:

Guard	**Value**
Minimum DC mean	50000
Maximum DC mean	262143 (exclusive)
Minimum AC	1000
Minimum perfusion index	0.10
Firmware HR acceptance	30..240 bpm
Bridge HR validation	20..250 bpm
SpO2 acceptance	>0..100 %

If quality is poor or the library algorithm does not mark a result valid, status becomes `low_quality` and the value is not published as valid. The 100-sample window is an optical estimate, not a clinical measurement.

C.7 MPU-6050/MPU-6500 path

`initMpuRaw()`:

- checks `WHO_AM_I` `0x68` or `0x70`;
- resets power management (`0x80`, delay, `0x00`);
- rejects a sleep bit set (`0x40`);
- sets `SMPLRT_DIV=0x04` (≈200 Hz with the base rate), `CONFIG=0x03` (44 Hz DLPF), `GYRO_CONFIG=0x00` (±250 dps), `ACCEL_CONFIG=0x10` (±8 g);
- verifies accel range bits `0x18 == 0x10` and gyro range bits `0x18 == 0x00`;
- attempts addresses `0x68`, then `0x69`.

`updateMpu()` reads 14 bytes starting at `ACCEL_XOUT_H`, converts three accelerometer and three gyroscope words, ignores the on-die temperature bytes, subtracts calibration biases when valid, and computes:

- `currentMotionG = magnitude3(ax, ay, az);`
- `currentGyroDps = magnitude3(gx, gy, gz);`

- orientation and stationary/moving state;
- a simple step candidate when |magnitude - 1 g| crosses 0.40 g with a 380 ms guard;
- fall candidate: free fall <0.6 g, impact >2.5 g after 60–600 ms, active for 6 s.

Conversion constants:

accel_g = raw / 4096.0

gyro_dps = raw / 131.0

Compile-time assertions verify ±4096 counts = ±1 g, 2048 = 0.5 g, 131 = 1 dps, and -262 = -2 dps.

C.8 MPU calibration — honest limits

calibrateMpu() tries up to two 80-sample windows at 50 Hz. A window is stationary only if:

- each accel axis range ≤ 0.15 g;
- each gyro axis range ≤ 4.0 dps;
- mean acceleration magnitude is between 0.7 and 1.3 g.

If stationary, it records gyro bias. For accelerometer bias, it requires one dominant axis near ±1 g and other axes within 0.20 g; it then computes a single-orientation bias. If orientation is unknown, gyro-only calibration completes.

This is **not** a six-face accelerometer calibration, not a temperature-compensated calibration and not a clinical motion validation. It only improves startup bias under one resting orientation.

D. Technical decisions and trade-offs

Decision	Rationale	Limitation
Checked raw I2C for TMP117	A library call/exception cannot silently become a temperature; raw diagnostic matches firmware path	More code; physical bus still must be validated
Scan four TMP117 addresses	ADD strapping is not assumed; prior observation was 0x49	A responding device with wrong WHO_AM_I is rejected, not accepted

Decision	Rationale	Limitation
Require WHO_AM_I=0x0117	Prevents an address-compatible but wrong device from being read as TMP117	A failed identity check needs electrical investigation
Configuration 0x01A0 + readback	8-sample averaging at 500 ms balances noise and bus traffic; readback catches write failure	Accuracy still requires reference validation
Read config before temperature	Reading config clears DATA_READY; checking readiness prevents stale-result reuse	Adds one bus transaction per sample
Reject 0x0000/0xFFFF	Canonical stuck-bus/all-ones patterns must not look like near-zero measurements	A genuinely stuck reading is unavailable, not "0 °C"
Accept -55..150 °C for actual sensor	This is the TMP117 code range, not a clinical range	Acceptance is not accuracy; package temperature may differ from target
Confirm >5 °C jumps	Suppresses a single corrupt/spurious code	Delays a genuine rapid change by one sample
No fixed offset	Wrist-to-core offsets are not clinically defensible; nonzero hardware offset is reported only	Absolute temperature interpretation still needs calibration/reference
Actual sensor field separate from wrist	Prevents package temperature from becoming a clinical wrist reading	Clinical wrist remains unavailable until contact is verified
Visual/optical quality thresholds	Reject DC-only, saturated or low-perfusion windows before publishing	Engineering thresholds, not clinical accuracy criteria
±8 g / ±250 dps ranges	Enough range for motion and fall candidates; good resolution	Saturation and sign/axis behavior still need physical testing
Single-orientation MPU bias	Practical startup calibration	Not full six-face; orientation-dependent error remains
No SHT40/MLX in active build	They are not installed; counting them would	Legacy code is still present but disabled; never present it

Decision	Rationale	Limitation
	misrepresent inventory	as tested

D.1 Code range versus accuracy — say it precisely

- `-55..150 °C` is the **accepted TMP117 code range** used by the source.
- `-10..60 °C` is the **clinical wrist range** that a future verified-contact path must also satisfy.
- Neither range is a statement of measured accuracy.
- No reference thermometer, controlled bath, ambient log or uncertainty budget was used in this audit.
- Do not say "the TMP117 is accurate to …" without a reference measurement and documented conditions.

E. Cross-module contributions and interfaces

E.1 Hardware evidence consumed by the bridge

`telemetry_bridge.py` independently enforces the same safety ideas:

- TMP117 raw must be an integer `0..0xFFFF`;
- valid actual sensor must be present, have raw/timestamp, be in `-55..150`, convert within `0.02 °C`, and not be `0x0000/0xFFFF`;
- non-valid actual sensor must have null value/raw/timestamp;
- clinical wrist with `unverified` contact must have status `unverified` and null value/raw/timestamp;
- a verified wrist value must match actual raw/value/timestamp and be in `-10..60`;
- a claimed SHT40/MLX presence or non-null legacy reading is rejected.

This is defense in depth: firmware checks and bridge checks are separate implementations.

E.2 Hardware evidence consumed by the dashboard

`DashboardCore.metricStatus()` checks:

- sensor presence;

- recognized status;
- valid TMP117 raw not 0/0xFFFF, value in -55..150, converted value within 0.02;
- for wrist, wrist_contact_status === "verified", status valid, raw conversion within 0.02, matching actual sensor raw/timestamp/value within 0.02, and wrist range -10..60;
- non-valid values with leaked evidence become sensor_error.

The dashboard can therefore hide a bad value even if a producer tries to send it, but it is not a substitute for firmware/bridge validation.

E.3 Hardware evidence consumed by storage and safety

- TelemetryStore.measurements() stores wrist only with verified contact, non-stuck raw, valid status, matching actual TMP117 evidence and range.
- temperature_safety.evaluate_temperature_safety() masks invalid actual-sensor fields and keeps core/fever unavailable.
- The baseline cannot use wrist data while contact is unverified because storage produces null.

E.4 What Member 2 should hand to other members

1. Actual TMP117 is not wrist/core; range semantics are different.
2. The raw diagnostic is the source of truth for the unresolved anomaly.
3. MAX/MPU values are estimates with engineering quality guards, not clinical measurements.
4. No SHT40/MLX hardware may be presented as present.
5. Every hardware claim needs a date, board, wiring conditions and reference/measurement method.

F. Evidence classification for this area

Claim	Classification	Verification
TMP117 identity/config/read code exists	[SOURCE]	tmp117_protocol.h, initTmp117(), readTmp117()
Signed conversion/vectors	[TEST] + compile-time asserts	test_tmp117_vectors.py; firmware build

Claim	Classification	Verification
0xFFFF and 0x0000 rejected	[SOURCE] + [TEST]	header, firmware, Python/Node tests
Raw diagnostic exists and builds	[SOURCE] + [BUILD]	diagnostics/tmp117_raw/; fresh build
Prior address 0x49	[RECORDED HW OBS] only	Audit manifest; not reproduced
Prior raw 0x0ED3 → 29.6484375	[RECORDED HW OBS] only	Audit manifest; not reproduced
Prior 60 reads/0 failures/0 stuck	[RECORDED HW OBS] only	Audit manifest; not reproduced
-0.01 °C cause	**Hypothesis only**	Raw diagnostic log still required
TMP117 accuracy	[UNVERIFIED HW]	No reference thermometer
MAX30102 accuracy	[UNVERIFIED HW]	No optical/reference validation
MPU axes/calibration/fall behavior	[UNVERIFIED HW]	No physical test
OLED readability	[UNVERIFIED HW]	No visual inspection
SHT40/MLX operation	**Not applicable**	Not installed, compiled out

F.1 Fresh build evidence

- Main firmware: **SUCCESS**, RAM 21,424 / 327,680 bytes, flash 326,001 / 6,553,600 bytes.
- TMP117 diagnostic: **SUCCESS**, RAM 18,916 / 327,680 bytes, flash 282,377 / 6,553,600 bytes.
- Fresh checks were run on 2026-10-29.
- Build success proves compilation/linking for the configured target, not electrical correctness.

G. Debugging: hardware-focused scenarios

G.1 The -0.01 °C anomaly

Observed in prior audit records: a display around -0.01 °C; raw 0xFFFF converts to -0.0078125 °C, which two-decimal formatting rounds to -0.01 °C.

What is established: the numerical relationship; the firmware rejects 0xFFFF; the diagnostic

would flag all-ones.

What is not established: that the actual event was caused by 0xFFFF, or by a released bus, mis-address, incomplete read, power issue or another fault.

Debugging steps:

1. Upload diagnostics/tmp117_raw to the actual board.
2. Capture at least five minutes, including the event.
3. Record address, WHO_AM_I, config before/after, writable-mask match, offset, raw MSB/LSB, drdy, failures, all-ones/all-zeros counts.
4. If 0xFFFF/0x0000 appears, inspect power, ground, pull-ups, solder joints, address strapping and GPIO 8/9.
5. Do not "fix" the display with an offset.
6. Only then update the claim from hypothesis to observed cause.

G.2 TMP117 not responding or wrong identity

Checklist:

- Is VIN 3.3 V (not 5 V), with common ground?
- Is SDA on GPIO 8 and SCL on GPIO 9?
- Are pull-ups appropriate for the bus?
- Does the scan ACK a candidate in 0x48–0x4B?
- Does WHO_AM_I read 0x0117?
- Does the configuration readback match the writable mask of 0x01A0?
- Is there an address conflict, e.g. another fixed 0x48 device?
- Is the offset register readable? A nonzero offset is reported, not silently applied.

A responding address is not sufficient; identity and configuration must match.

G.3 MAX30102 "no finger" versus sensor failure

The code distinguishes:

- ir < 50000 or red < 50000 → no contact (not_available);
- >= 262143 or poor AC/PI → low quality;
- no FIFO samples at all after initialization → sensor_error/no_fifo_samples.

Debugging should check power, I2C, part ID, FIFO configuration, finger placement and ambient light. Do not lower thresholds arbitrarily just to make a number appear.

G.4 MPU address instability or calibration failure

- If AD0 floats, the address can switch between 0x68/0x69; the firmware probes both but a stable physical strap is required.
- If WHO_AM_I is neither 0x68 nor 0x70, initialization fails.
- If the device moves during the 80-sample window, calibration fails (device did not remain stationary); retry with the device still.
- A "basic_complete" result is single-orientation bias, not a full calibration.
- Verify signs, axes and ranges by rotating the device in known directions; do not infer them from a single stationary reading.

G.5 Reference measurement protocol (not yet performed)

For a defensible TMP117 accuracy claim, record:

- board and sensor identifier;
- supply voltage and bus wiring;
- sensor address and configuration;
- ambient temperature and airflow;
- contact/stabilization time;
- reference instrument and its calibration/uncertainty;
- multiple setpoints and repeated readings;
- raw values, converted values and error;
- offset register value and whether it was applied;

- uncertainty budget.

Without this, the source range `-55..150 °C` is only a code acceptance range.

H. Testing and verification

H.1 Host/software evidence

Test/check	What it covers	Result
tests/test_tmp117_vectors.py	Signed conversion, byte order, stuck patterns, range vectors	Passed as part of 62 Python tests
src/tmp117_protocol.h asserts	Conversion/stuck/range helpers at compile time	Passed in firmware/diagnostic builds
src/sensor_math.h asserts	MPU scale and MAX quality guard relationships	Passed in firmware build
Temperature tests	Actual vs unverified/verified wrist, raw mismatch, fever policy	Passed
History tests	Unverified wrist not stored; invalid/missing not zero	Passed
Dashboard tests	Stuck/missing/inconsistent evidence masked	Passed
Both PlatformIO builds	Firmware/diagnostic compile and link	SUCCESS on 2026-10-29

H.2 Required physical tests

1. **TMP117:** raw diagnostic ≥5 minutes; identity/config/offset/raw/failure summary; reference comparison under controlled conditions.
2. **MAX30102:** no finger, steady finger, motion, removal, ambient light; record HR/SpO2 and quality against a reference.
3. **MPU:** rest, known orientations, each axis rotation, range/sign checks, failed-read behavior, safe fall simulation.
4. **OLED:** legibility, contrast, text overflow, emergency banner.
5. **SOS:** press/cancel/auto-clear, serial/OLED/dashboard propagation.

6. **Hot-plug:** remove/reconnect TMP117/MAX/MPU and confirm null/status/recovery.

7. **End-to-end:** assembled device, serial bridge, dashboard, history and alert paths.

H.3 What the tests do not prove

They do not prove that a sensor is present, that its package is in contact with the intended site, that the raw bus is electrically clean, that the optical estimate is accurate, or that the device is safe for clinical use.

I. HOD preparation

I.1 60–90 second explanation

"My allocated area is hardware and sensor reliability. The installed measurement set is a MAX30102 optical sensor, an MPU-6050 or MPU-6500, and a TMP117. The TMP117 is on the primary bus with the OLED and MAX30102 at 50 kHz; the MPU has a dedicated 400 kHz bus. For the TMP117 I use checked raw I2C, not a library. The temperature register is signed 16-bit big-endian with 0.0078125 degrees per LSB. I verify WHO_AM_I 0x0117, configure continuous conversion with eight-sample averaging at 500 milliseconds, check the writable-mask readback, and read the configuration before the temperature because that read clears DATA_READY. Raw 0x0000 and 0xFFFF are rejected as stuck-bus patterns, and a jump over five degrees needs a confirming sample. The actual TMP117 reading is not the clinical wrist value: the wrist value stays null and unverified because no contact detector has been validated. MAX30102 and MPU data have engineering quality/range checks but no reference validation. The builds pass; the physical sensors and accuracy are still unverified."

I.2 Technical questions and answers

1. **What sensors are installed, and what is not installed?** MAX30102, MPU-6050/MPU-6500, TMP117, SSD1306 and an SOS button. SHT40 and MLX90614 are not installed, compiled out and not counted.

2. **Why is the TMP117 on the same bus as the OLED and MAX30102?** They are all 3.3 V I2C devices on the primary `Wire` bus at 50 kHz; the IMU has a separate 400 kHz bus to isolate its faster traffic. The actual address is verified in `0x48–0x4B`.

3. **What is TMP117's identity and configuration?** `WHO_AM_I` register `0x0F` must be `0x0117`. Configuration `0x01A0` selects continuous conversion, eight-sample averaging and a 500 ms

cycle; the writable-mask readback must match.

4. **How does the signed conversion work?** Interpret the 16-bit big-endian code as signed (int16_t) and multiply by 0.0078125 °C/LSB. 0xFFFF becomes -0.0078125, not a large positive number.

5. **Why reject 0xFFFF and 0x0000?** They are canonical all-ones/all-zeros patterns indicating a released, mis-addressed or incompletely read bus. Accepting them would create a fake near-zero temperature.

6. **Why read configuration before temperature?** Reading configuration clears the DATA_READY bit. The firmware checks busy/data-ready first so it does not reuse an old conversion or read before a result exists.

7. **What happens on a large temperature jump?** A change over 5 °C is held as low_quality pending confirmation. A second sample within 0.5 °C allows the new value; otherwise it remains untrusted.

8. **Is the TMP117 reading a wrist or core temperature?** No. It is the actual sensor-package reading. It can be valid outside -10..60 °C because its accepted code range is -55..150 °C, but it is never promoted to wrist or core.

9. **What must a future verified wrist reading prove?** Verified contact, valid wrist status, non-stuck raw, timestamp, signed conversion within 0.02 °C, matching actual-sensor value/raw/timestamp, and wrist value within -10..60 °C. No fixed offset is allowed.

10. **What does the MAX30102 quality guard check?** Mean DC 50000..262143, AC at least 1000, perfusion index at least 0.10, plus the library's validity flags. These are engineering guards, not clinical thresholds.

11. **How are MPU values scaled, and what are the ranges?** ±8 g accelerometer with 4096 LSB/g; ±250 dps gyroscope with 131 LSB/dps; 14-byte read starts at ACCEL_XOUT_H. The on-die temperature bytes are ignored.

12. **What does MPU calibration actually do?** It estimates gyro bias and a single-orientation accelerometer bias from 80 stationary samples. It is not a six-face calibration and not a clinical motion validation.

13. **What is the TMP117 diagnostic for?** It captures raw identity/configuration/offset/raw/failure evidence independently of the main firmware, so the -0.01 °C anomaly can be confirmed or refuted.

14. **Why is sensor accuracy still unverified?** No reference thermometer, optical reference,

controlled conditions or assembled-device test were available. Builds and host tests cannot prove physical accuracy.

I.3 Five deeper follow-up questions

1. **"Why not simply use the Adafruit TMP117 library?"** The checked raw path gives explicit control over ACK/identity/config/data-ready, rejects stuck patterns, and matches the raw diagnostic. A library abstraction could still be safe, but it would need equivalent evidence; the project chose explicit transactions for auditability.

2. **"What is the difference between a code range and an accuracy specification?"** The range is what the solver accepts (`-55..150 °C`); accuracy is how close a reading is to a reference under stated conditions. The project has no reference data, so it cannot claim a measured accuracy.

3. **"Why not calibrate the TMP117 with an offset?"** The current design reports a nonzero hardware offset but does not apply it. A fixed offset is not a validated calibration and could hide a fault. Any calibration would need traceable reference measurements and documented conditions.

4. **"Can the MPU detect falls reliably?"** Not established. It uses a two-stage free-fall/impact heuristic with fixed thresholds. Sensitivity/specificity, placement, false positives and false negatives have not been measured.

5. **"What would you test first on real hardware?"** The TMP117 raw diagnostic, because it resolves identity/config/raw/failure ambiguity and the known anomaly. Then MAX/MPU and end-to-end alert paths.

I.4 Two cross-module questions

1. **"The firmware sends actual TMP117 data with wrist unverified. What does the backend do?"** `_validate_temperature()` accepts the actual fields only with matching raw/timestamp/range; it requires wrist status `unverified` and wrist value/raw/timestamp null. The dashboard shows actual and wrist separately; core/fever remain unavailable.

2. **"How does the bridge prevent a stuck TMP117 raw code from entering history?"** `_validate_temperature()` rejects `0x0000`/`0xFFFF` when marked valid, requires conversion within `0.02 °C`, and `TelemetryStore.measurements()` repeats raw/range/status checks before storing the clinical wrist metric. Actual TMP117 is not stored as wrist history.

I.5 Three improvements

1. **Capture and archive the TMP117 raw diagnostic on the assembled board**, including the anomaly window, and link the log to the audit record.
2. **Add a controlled reference-thermometer procedure for TMP117 and a reference/optical protocol for MAX30102**, with uncertainty and environmental conditions.
3. **Implement and validate a wrist-contact detector before exposing clinical wrist values**; until then keep the unverified/null policy. Separately improve MPU calibration/testing beyond single-orientation bias.

I.6 Ten key facts

1. Installed measurement inventory is MAX30102, MPU and TMP117; SHT40/MLX are absent.
2. Primary I2C is GPIO 8/9 at 50 kHz; IMU is GPIO 17/18 at 400 kHz.
3. TMP117 identity is 0x0117; configuration 0x01A0; conversion is signed × 0.0078125.
4. 0xFFFF converts to -0.0078125 °C, which rounds to -0.01 °C on a two-decimal display.
5. 0x0000 and 0xFFFF are rejected as stuck-bus patterns.
6. Actual sensor range is -55..150 °C; clinical wrist range is only relevant with verified contact and is -10..60 °C.
7. Clinical wrist values are currently always null/unverified; no fixed offset is applied.
8. MAX quality guards are DC 50000..262143, AC ≥ 1000, PI ≥ 0.10.
9. MPU uses ±8 g / ±250 dps; calibration is single-orientation bias, not six-face.
10. Fresh builds succeed; physical sensor accuracy and the TMP117 anomaly cause remain unverified.

I.7 Five terms

Term	Explanation
WHO_AM_I	TMP117 identity register 0x0F; expected 0x0117.
DATA_READY	Configuration status bit 0x2000 indicating a conversion is available; reading configuration clears it.

Term	Explanation
Stuck-bus pattern	Raw 0x0000 or 0xFFFF that indicates an untrustworthy bus/read rather than a real temperature.
Perfusion index	(maximum − minimum) / mean × 100 for the optical window; an engineering signal-quality measure, not a clinical diagnosis.
Single-orientation bias	A startup accelerometer bias estimated with one resting orientation; not a full six-face calibration.

I.8 Three source files to open first

1. src/tmp117_protocol.h — TMP117 registers, conversion, stuck/range helpers and compile-time vectors.
2. src/main.cpp — initTmp117(), readTmp117(), initMax30102(), finishOpticalWindow(), initMpuRaw(), calibrateMpu(), updateMpu().
3. diagnostics/tmp117_raw/src/main.cpp — standalone raw diagnostic and summary counters.

I.9 Three things never to claim without evidence

1. **"TMP117 is accurate/calibrated."** No reference measurement, controlled conditions or uncertainty analysis exists.
2. **"The -0.01 °C issue is definitely caused by 0xFFFF."** That is a numerical hypothesis; the raw diagnostic log is still required.
3. **"MAX30102/MPU/OLED/fall detection works on the device."** Software paths/builds exist; physical validation is missing.

I.10 Responsibility checklist

- ☐ I can list every installed and non-installed component.
- ☐ I can draw/explain both I2C buses and addresses.
- ☐ I can derive the TMP117 conversion from raw bytes.
- ☐ I can explain WHO_AM_I, 0x01A0, DATA_READY, writable-mask readback and offset.

- ☐ I can explain stuck-bus and abrupt-change rejection.
- ☐ I can distinguish actual sensor range, clinical wrist range and accuracy.
- ☐ I can explain why wrist is unverified/null and why no fixed offset is used.
- ☐ I can explain MAX DC/AC/PI guards and their limits.
- ☐ I can explain MPU scale constants and calibration limits.
- ☐ I can describe the TMP117 diagnostic capture procedure.
- ☐ I can label prior observations as not reproduced.
- ☐ I can state the three "never claim" boundaries without prompting.

J. Honest boundaries

J.1 What this member can defend

- The installed inventory and wiring/bus design.
- The TMP117 protocol/conversion/identity/config checks and raw diagnostic.
- Why stuck values, out-of-range values and unconfirmed jumps are rejected.
- The actual-sensor versus clinical-wrist separation and the no-offset policy.
- MAX/MPU engineering guards and their limitations.
- The precise gaps in physical validation.

J.2 What this member must not claim

- That any sensor is accurate without a reference measurement.
- That the prior `0x49` address, raw values or 60-read observation were reproduced in this pass.
- That SHT40/MLX90614 are installed or working.
- That the actual TMP117 reading is wrist or core temperature.
- That the MPU calibration is a full calibration or a validated fall detector.
- That the device is clinically safe or an emergency service.

- That the OLED displays the exact long labels.
- That firmware/build success proves electrical correctness.

J.3 Safe closing sentence

"The sensor code is implemented with explicit identity, range, raw-evidence and stuck-pattern checks, and it builds. What I cannot claim yet is physical accuracy, contact verification or clinical validity; those require the assembled board, a reference and a documented protocol."